



Assessment Cover Sheet

Assessment	Project(Individual)		
Assessment	Uncontrolled	Individual	Not must-pass
Due	11 December 2025	Course	IT9002
Course	Natural Language Processing		
Internal Moderator's	Dr. Abdelhameed Fawzy		
External Examiner's	Dr.		

Instructions:

1. This cover sheet must be completed (section in blue below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is XXX minutes/hours/days.
3. This assessment carries XXX marks distributed to a total of XXX questions assessing CILO X and CILO X.
4. The materials allowed for use in this assessment are XXX, XXX, and XXX.
5. The **use of generative AI tools is strictly prohibited**.
6. References consulted (if any) must be properly acknowledged and cited.
7. The assessment has a total of XXX pages.

Learn	1120115	Date	08/1/2026
Learner	Enas Nayel		
Progra	MSc in AI		
Progra	IT9002		
Lecture	Sini Raj Pulari		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Do not write beyond this line. For assessor use

Learn

Enas

Assessor's Name	Sini Raj Pulari		
Marking		Maks	

Comments:

Contents

5.....	The use of generative AI tools is strictly prohibited.	
1		
1120115		08/1/2026
1		
Assessor's Name		1
Project Overview		3
Motivation.....		3
Problem Statement / Project Definition		3
Expected Result		3
Task 2.....		3
Task 3.....		5
Task 4.....		7
Bag of words (BoW) and N-gram.....		7
N-gram		8
Part-of-speech tagging (POS)		8
Named Entity Recognition (NER)		9
Task 5.....		9
Task 6.....		10
Github Link.....		13
Dataset Link.....		13

Sentiment Tweet Analysis

Project Overview

Motivation

Social media platforms such as Twitter generate large volumes of real-time public opinion that can be valuable for decision-making. However, this information is difficult to interpret manually because tweets are short, informal, and often contain slang, abbreviations, emojis, and rapidly changing topics. In addition, the sentiment expressed in tweets can shift quickly depending on events, trends, or viral discussions. For organizations, being able to understand this sentiment early can support reputation monitoring, customer feedback analysis, and faster responses to public concerns. Therefore, using Natural Language Processing (NLP) to automate sentiment analysis is important because it enables scalable, consistent, and timely interpretation of large amounts of text data.

Problem Statement / Project Definition

Social media mentions of organizations often contain unclear, mixed, or rapidly changing sentiment, making it challenging for organizations to accurately interpret public perception in a timely manner. Manual analysis is not practical due to the high volume of tweets and the noisy nature of social media text. This project addresses this challenge by applying NLP techniques to a large Twitter sentiment dataset obtained from Hugging Face, with the goal of building an automated sentiment classification model. The model will learn patterns in tweet text and classify sentiment into binary categories (negative vs. positive), supporting faster and more reliable sentiment detection compared to manual review.

Expected Result

The expected outcome of this project is to develop and evaluate a sentiment classification system that can automatically predict the sentiment of tweets with reliable and balanced performance. Specifically, the target is to achieve an **F1-score of at least 70%**, ensuring that the model performs well across both classes and does not favor one sentiment category over the other. In addition, the project aims to demonstrate a complete NLP pipeline including text preprocessing, feature extraction (e.g., TF-IDF), model training, and evaluation using metrics such as accuracy, precision, recall, F1-score, and confusion matrix. This will confirm whether the developed approach can effectively support automated interpretation of public opinion on Twitter at scale.

Task 2

The dataset was selected from Hugging Face, a well-known and reputable source for NLP datasets. I searched the website specifically for a dataset suitable for achieving my objective, as outlined in my introduction, which emphasizes the need to analyze tweets. My primary criteria included finding a dataset designed for sentiment classification, containing tweets, having prior use by other researchers, and of sufficient size to support effective training. The chosen dataset that meets these requirements is the `gxb912/large-twitter-tweets-sentiment` dataset [1] [Figure 1] It is intended for sentiment classification, comprised of tweets, and has a sizeable volume. The dataset shape is (224,994, 2) [Figure 3, Figure 2] It has two labels (0 and 1) and the distribution is moderately imbalanced (0 = 94,837; 1 = 130,157).

```
[11]: merged_df.head()

[11]:
```

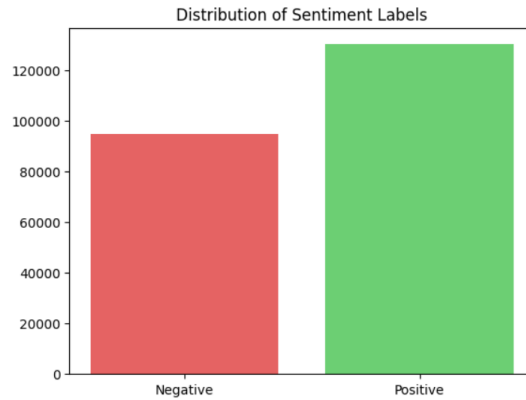
	sentiment	text
0	1	@tonigirl14 love you toooooo!! TG LOL Gngb
1	0	@jun6lee I told myself: Don't click on this li...
2	0	The man who rendered his voice to Mickey Mouse...
3	1	@Shontelle_Layne I think red would be nice. O...
4	0	@Silverlines - I guess. 'Cause one of her twee...

```
[12]: merged_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 224994 entries, 0 to 224993
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0    sentiment  224994 non-null  int64
1    text       224994 non-null  object
dtypes: int64(1), object(1)
memory usage: 3.4+ MB
```

Figure 1 Initial dataset preview (first 5 rows) showing the two columns: sentiment and text

```
[18]: #the Class balance
counts = merged_df['sentiment'].value_counts()
plt.bar(['Negative','Positive'], [counts[0], counts[1]], color=['#e56363','#6ccf73'])
plt.title('Distribution of Sentiment Labels')
plt.show()
```



```
[19]: # examine the percentage of other Language
# not language id for each text
```

Figure 3 Overview of class to check the balance between the number of negative observations and positive observations, but it shows moderate imbalance.

```
[14]: merged_df.shape
[14]: (224994, 2)
[15]: merged_df.dtypes
[15]: sentiment    int64
      text      object
      dtype: object
[16]: merged_df['sentiment'].unique()
[16]: array([1, 0])
[17]: merged_df['sentiment'].value_counts()
[17]: sentiment
      1    130157
      0     94837
      Name: count, dtype: int64
```

Figure 2 Dataset information summary showing 224,994 entries, two columns, (sentiment as int64 and text as object).

Task 3

In my project, as part of the NLP pipeline, before applying preprocessing and feature extraction, I cleaned the dataset to reduce noise commonly found in tweets, such as hashtags, numeric values, slang words, word elongation, extra whitespaces, and user mentions. These elements can introduce non-informative tokens, increase vocabulary size, create sparsity in TF-IDF / Bag of Words features, and reduce feature quality[2]. Hashtags were handled by removing the symbol and cleaning the remaining text, numeric values were removed when they did not contribute to sentiment meaning, slang words were replaced using a slang dictionary dataset containing common abbreviations and their standard forms [3], and elongated words were normalized (e.g., “coool” → “cool”) by reducing repeated characters. Tweets also often include mentions (e.g., @username), which were removed before tokenization to reduce vocabulary sparsity and improve feature consistency [Figure 4, Figure 5].

After cleaning, tokenization was applied to split the text into words (tokens)[Figure 6]. Then lemmatization was implemented to reduce tokens to their root form so that different word forms are treated as one base term (e.g., “years” to “year”)[Figure 7]. Previous studies show that the effectiveness of stemming and lemmatization depends on the NLP task and dataset characteristics. Therefore, in this project (tweet sentiment classification using TF-IDF features), lemmatization was selected to reduce word-form variation while preserving valid root words, improving interpretability and limiting distortion compared to aggressive stemming. This helps reduce feature dimensionality and supports better generalization. Finally, TF-IDF was applied for feature extraction, where terms are weighted based on their frequency within a tweet and their rarity across the dataset [Figure 8]. As a result, very common terms receive lower weight, while more informative terms receive higher weight.

```
#visualizaion the Tweet
def tweet(data, m, n):
    print('-''Tweet before preprocessing')
    for i in range(n - m + 1):
        print('-', m+1, '-----')
        print(data.iloc[m])
        m += 1

tweet(merged_df['text'], 0, 10)
```

```
-Tweet before preprocessing
- 1 -----
@tonigirl14 love you toooooo!! TG LOL Gngb
- 2 -----
@jun6lee I told myself: Don't click on this link. But I just did. Boohooo
- 3 -----
The man who rendered his voice to Mickey Mouse expired. He was 62 years old! i'm a Mickey fan!
- 4 -----
@Shontelle_Layne I think red would be nice. Or maybe you could do a mix of red and black.
- 5 -----
@Silverlines - I guess. 'Cause one of her tweets says she is confused.
- 6 -----
Have to go home in awhile. Goodbye, internet. I will miss you.
- 7 -----
@lomolibertine *faints* I want!! Everything looks good when its put through a fisheye haha
- 8 -----
```

Figure 4 Text before cleaning

```
#visualizaion the Tweet
def tweet(data, m, n):
    print('-''Tweet after preprocessing')
    for i in range(n - m + 1):
        print('-', m+1, '-----')
        print(data.iloc[m])
        m += 1

tweet(df['ctext'], 0, 10)
```

```
-Tweet after preprocessing
- 1 -----
love you too T G L O L Gngb
- 2 -----
I told myself Do not click on this link But I just did Boohoo
- 3 -----
The man who rendered his voice to Mickey Mouse expired He was years old i am a Mickey fan
- 4 -----
I think red would be nice Or maybe you could do a mix of red and black
- 5 -----
I guess Cause one of her tweets says she is confused
- 6 -----
Have to go home in awhile Goodbye internet I will miss you
- 7 -----
faints I want Everything looks good when its put through a fisheye haha
- 8 -----
```

Figure 5 Text after cleaning

```
In [148... # Initialize NLTK TweetTokenizer with Twitter-specific settings
tt = TweetTokenizer(preserve_case=False, strip_handles=True, reduce_len=True)
# Tokenize cleaned tweet text into individual word tokens
df["tokens"] = df["ctext"].fillna("").astype(str).apply(tt.tokenize)

In [149... #check the tokenizer
df["tokens"].head()

Out[149... 0          [love, you, too, t, g, l, o, l, gngb]
1    [i, told, myself, do, not, click, on, this, li...
2    [the, man, who, rendered, his, voice, to, mick...
3    [i, think, red, would, be, nice, or, maybe, yo...
4    [i, guess, cause, one, of, her, tweets, says, ...
Name: tokens, dtype: object
```

Figure 6 Tokenization

```
In [156... # Lemmatizer
lemmatizer = WordNetLemmatizer()

print("{0:20}{1:20}".format("--Word--", "--Lemma--"))

for token_list in df["ftokens"].head(10):
    if isinstance(token_list, list):
        for word in token_list:
            if isinstance(word, str):
                print("{0:20}{1:20}".format(word, lemmatizer.lemmatize(word)))
    elif isinstance(token_list, str):
        print("{0:20}{1:20}".format(token_list, lemmatizer.lemmatize(token_list)))

--Word--      --Lemma--
love          love
told          told
not           not
click        click
link         link
man          man
rendered     rendered
voice        voice
mickey       mickey
mouse        mouse
expired      expired
years        year
old          old
mickey       mickey
fan          fan
think        think
red          red
would        would
nice         nice
```

Figure 7 Lemmatization

```

neg = {"no", "not", "nor", "never"}
stop_words = list(set(stopwords.words("english")) - neg)
tfidf = TfidfVectorizer(
    token_pattern=r"(?u)\b[a-z]{2,}\b",
    stop_words=stop_words,
    ngram_range=(1,2),
    min_df=10,
    max_df=0.9,
    sublinear_tf=True
)

tfidf_t = tfidf.fit_transform(df["ftext"])
y = df["sentiment"]
print(tfidf_t.shape)
print(tfidf.get_feature_names_out()[:20])

(197517, 18622)
['aahh' 'aages' 'aah' 'aahh' 'aamp' 'aand' 'aargh' 'aaron' 'aaw' 'aaww'
 'abandoned' 'abby' 'aberddeen' 'ability' 'abit' 'able' 'able afford'
 'able get' 'able help' 'able make']

tfidf_t.shape

```

Figure 8 TF-IDF

Task 4

After cleaning and preprocessing the tweets, the next step is to represent the text in a numerical form so that the model can learn from it. Since machine learning models cannot work directly with raw text, text representation is needed to convert tweets into features. In this project, I used three text representation techniques that are suitable for tweet sentiment analysis: Bag of Words, N-grams, and Word Embeddings. These techniques help capture important words, important word combinations, and the meaning relationship between words. The outputs and figures are provided to show how each technique represents the tweets.

Bag of words (BoW) and N-gram

In this project, Bag of Words representation was implemented using CountVectorizer to convert tweets into numerical feature vectors. The vectorizer was configured with `ngram_range=(1,2)` to capture both unigrams and bigrams, since sentiment can depend on single words as well as short phrases (e.g., “not good”). To reduce noise and sparsity, `min_df=10` was used to remove very rare terms, and `max_df=0.9` was used to remove extremely frequent terms that are not informative. The resulting feature matrix shape was (197,517, 18,623), meaning 197,517 tweets were represented using 18,623 vocabulary features, and the vocabulary size was 18,623 [Figure 9].

In [170...

```

#bag of words + Ngram Bigram
bow_vectorizer = CountVectorizer(
    ngram_range=(1, 2),
    min_df=10,
    max_df=0.9
)

X_bow = bow_vectorizer.fit_transform(df["ftext"])
y = df["sentiment"]
print("BoW matrix shape:", X_bow.shape)
print("Vocab size:", len(bow_vectorizer.get_feature_names_out()))
print("First 20 vocab words:", bow_vectorizer.get_feature_names_out()[:20])

```

```

BoW matrix shape: (197517, 18623)
Vocab size: 18623
First 20 vocab words: ['aahh' 'aages' 'aah' 'aahh' 'aamp' 'aand' 'aargh' 'aaron' 'aaw' 'aaww'
 'abandoned' 'abby' 'aberddeen' 'ability' 'abit' 'able' 'able afford'
 'able get' 'able help' 'able make']

```

Figure 9 CountVectorizer (unigrams + bigrams) output showing BoW feature matrix shape (197,517 × 18,623) and vocabulary size (18,623).

N-gram

N-grams are an extension of Bag of Words, where the model does not only use single words, but also uses word combinations such as two words together (bigrams). This is important in sentiment analysis because the sentiment meaning can change based on the phrase, especially with negation (for example, “not good” is different from “good”). In this project, bigrams were extracted and visualized to show the most frequent bigram phrases in the dataset [Figure 10]. The plot shows that several common bigrams include negation patterns such as “not know”, “not get”, “not even”, and “not good”, which supports why using bigrams is useful for tweets because it captures short phrases that Bag of Words alone may miss. This helps the model learn more accurate sentiment patterns from tweet text.

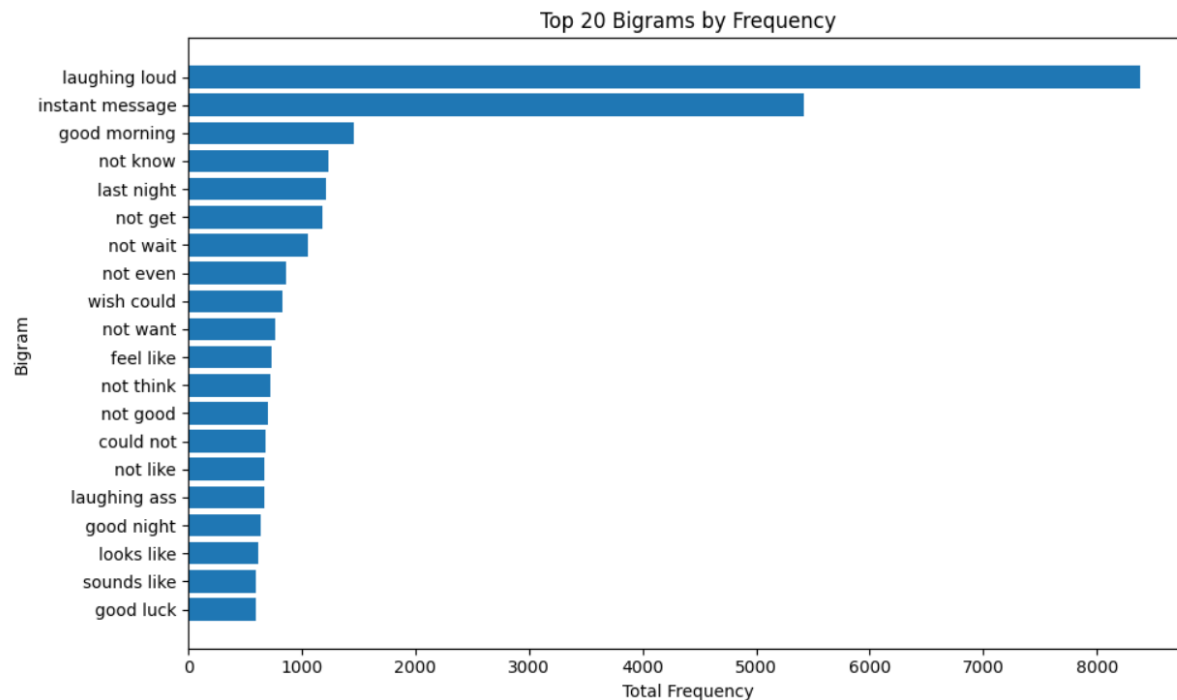


Figure 10 Top bigrams

Part-of-speech tagging (POS)

POS tagging assigns a grammatical tag to each token, such as noun (NN), verb (VBD), adjective (JJ), and adverb (RB). This helps describe how words are used inside the tweet, and it supports sentiment analysis because adjectives and adverbs often carry strong sentiment meaning (e.g., “good”, “bad”, “very”). In this project, POS tagging was applied to the tokenized tweets, and the output shows examples of tokens with their POS tags [Figure 11]. This representation helps in understanding the type of words that appear frequently in tweets and can support feature engineering or analysis of sentiment-carrying words.

```
In [166... #POS (Part-of-Speech)
def named_entity_recognition(pos_tags):
    return ne_chunk(pos_tags)

In [167... #apply the function
df["pos_tags"] = df["ftokens"].apply(pos_tag)

In [168... df["pos_tags"].head()

Out[168... 0          [(love, NN)]
1  [(told, VBD), (not, RB), (click, JJ), (link, NN)]
2  [(man, NN), (rendered, VBD), (voice, NN), (mic...)
3  [(think, NN), (red, VBD), (would, MD), (nice, ...)
4  [(guess, NN), (cause, NN), (one, CD), (tweets,...
Name: pos_tags, dtype: object
```

Figure 11 Part of speech (POS)

Named Entity Recognition (NER)

Named Entity Recognition (NER) is used to detect named entities in text such as people, organizations, and locations. This is useful for tweets because tweets often mention brands, public figures, or places, and sentiment can be related to these entities. In this project, NER was applied after POS tagging using `ne_chunk`, and the output is shown as chunked trees for sample tweets [Figure 12]. This helps identify which entities appear in tweets and supports analysis of sentiment around specific targets (for example, sentiment toward a company name or a person).

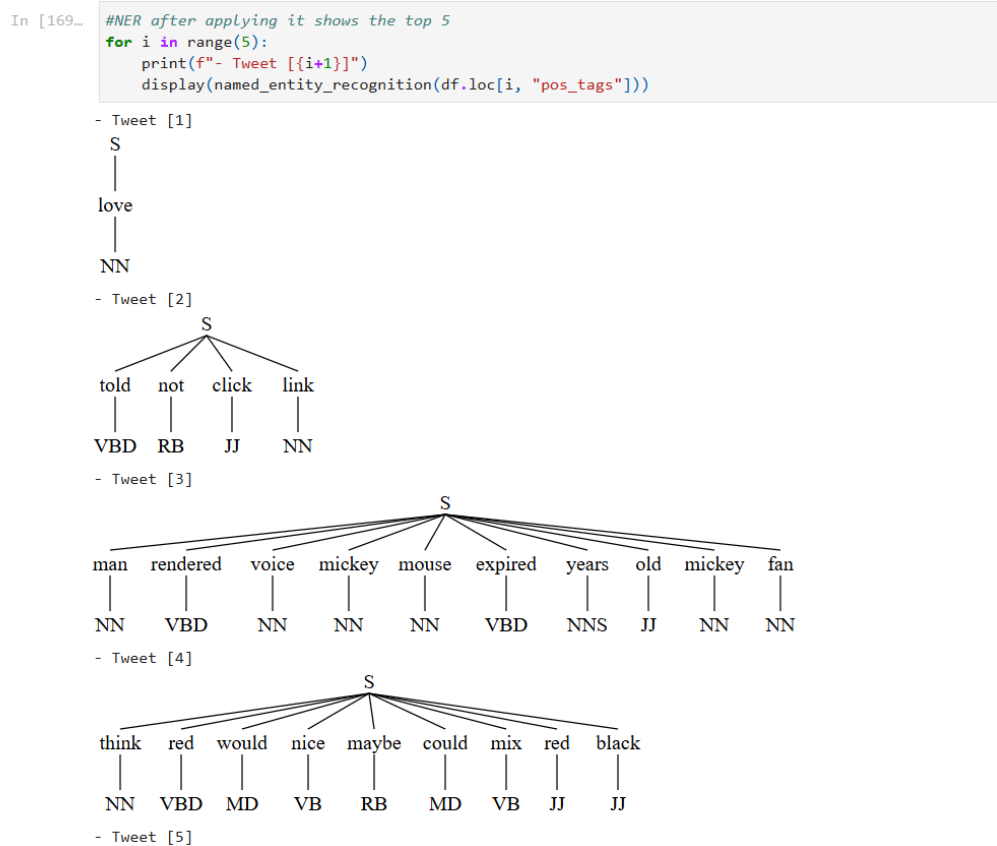


Figure 12 Named Entity Recognition (NER)

Task 5

After converting the tweets into a proper text representation (cleaned text, tokens, lemmatized text, and TF-IDF features), the next step was selecting suitable text classification methods for the business problem, which is binary sentiment classification (negative vs. positive). Since the dataset is large and tweets are short and noisy, both traditional machine learning models and transformer-based pretrained models were used. Traditional models work well with TF-IDF features, while transformer models are better at capturing contextual information in text.

Logistic Regression was selected as a baseline machine learning model because it is commonly used for text classification and performs well with high-dimensional and sparse features such as TF-IDF. It is simple, efficient, and provides stable performance on large datasets. Previous studies show that Logistic Regression performs well for sentiment analysis when combined with TF-IDF features, including on social media text[4]. Based on this, Logistic Regression was used as a reliable baseline model in this project.

Support Vector Machine (SVM) was selected because prior research shows that it performs better than Naïve Bayes for sentiment analysis tasks. Studies have reported that SVM achieves higher accuracy and better average performance due to its strong generalization ability [5]. SVM works by constructing an optimal separating hyperplane between sentiment classes, which makes it effective for text classification, especially for short and noisy text such as tweets. Therefore, SVM was selected as a stronger traditional machine learning model alongside Logistic Regression.

BERT was selected because it is a transformer-based model that captures bidirectional contextual information, which is important for understanding sentiment in tweets. Previous studies have shown that BERT-based models significantly improve sentiment classification performance compared to traditional machine learning approaches [7], [8] Figure 13. Due to its ability to model context and handle complex language patterns, BERT was used as a pretrained deep learning model in this project.

RoBERTa was selected because it improves upon BERT by using optimized pretraining strategies and larger training data, resulting in stronger language representations. Research has shown that RoBERTa-based models achieve

high performance for sentiment analysis tasks and outperform traditional machine learning models for Twitter sentiment analysis [6]. Based on this evidence, RoBERTa was selected to evaluate improved contextual modeling for tweet sentiment classification.

BERTweet was selected because it is pretrained specifically on Twitter data, making it more suitable for handling slang, abbreviations, emojis, and informal language commonly found in tweets. Studies have shown that Twitter-specific pretrained models perform better than general language models for sentiment analysis on tweets [7]. Based on this, BERTweet was selected as a domain-adapted transformer model for this project.

Using both traditional machine learning models and transformer-based models allows a fair comparison between feature-based and context-based approaches for tweet sentiment analysis.

Model 1

```
model_bert = pipeline(  
    "text-classification",  
    model="Praveen-R/Tweet_sentiment_analysis_with_BERT",  
    device=-1  
)
```

Device set to use cpu

```
res_bert = model_bert(X_test.tolist(), batch_size=32, truncation=True)
```

```
label_map_bert = {"LABEL_0": "negative", "LABEL_1": "neutral", "LABEL_2": "positive"}  
y_pred_bert_3 = [label_map_bert[r["label"]] for r in res_bert]
```

```
y_pred_bert = ["positive" if p in {"positive", "neutral"} else "negative"  
               for p in y_pred_bert_3]
```

Figure 13 Implementation of a pretrained BERT model for tweet sentiment classification using the Hugging Face pipeline

Task 6

Model Evaluation

After training and applying multiple sentiment classification models, their performance was evaluated using standard classification metrics suitable for binary sentiment analysis. The evaluation focused on Accuracy, Precision, Recall, F1-score, and Confusion Matrix visualizations, which together provide a balanced understanding of model performance, especially for datasets with mild class imbalance.

Traditional machine learning models (Logistic Regression and SVM) were evaluated using TF-IDF features. Transformer-based models (BERT, RoBERTa, and BERTweet) were evaluated using pretrained pipelines, where model predictions were mapped into binary sentiment classes (positive vs. negative) to ensure consistency across all models.

Confusion matrices were used to visually inspect how well each model distinguished between positive and negative tweets, highlighting correct classifications and common misclassification patterns. In addition, F1-score was emphasized as a key metric because it balances precision and recall and is more reliable than accuracy alone for sentiment classification tasks.

Based on the confusion matrix results, Logistic Regression showed slightly better overall performance than SVM. Logistic Regression achieved a higher number of True Positives and fewer False Negatives, meaning it was better at correctly identifying positive sentiment tweets and missed fewer positive cases. However, it showed slightly lower performance in identifying negative sentiment compared to SVM. In contrast, SVM achieved higher True Negatives and fewer False Positives, indicating better identification of negative sentiment, but it missed more positive tweets due to higher False Negatives. Overall, Logistic Regression provided more balanced performance across both sentiment classes and achieved higher overall accuracy, while SVM performed better specifically for negative sentiment detection. For the transformer-based models, all three showed strong performance in identifying positive sentiment, but with differences in handling negative sentiment. BERTweet achieved very high True Positives and low False Negatives, showing strong detection of positive sentiment, but it also produced more False Positives, meaning some negative tweets were misclassified as positive. BERT showed weaker performance in identifying negative sentiment, with higher False Positives compared to the other transformer models. RoBERTa demonstrated the most balanced performance among the

transformer models, achieving high True Positives while maintaining lower False Positives and the lowest False Positive Rate

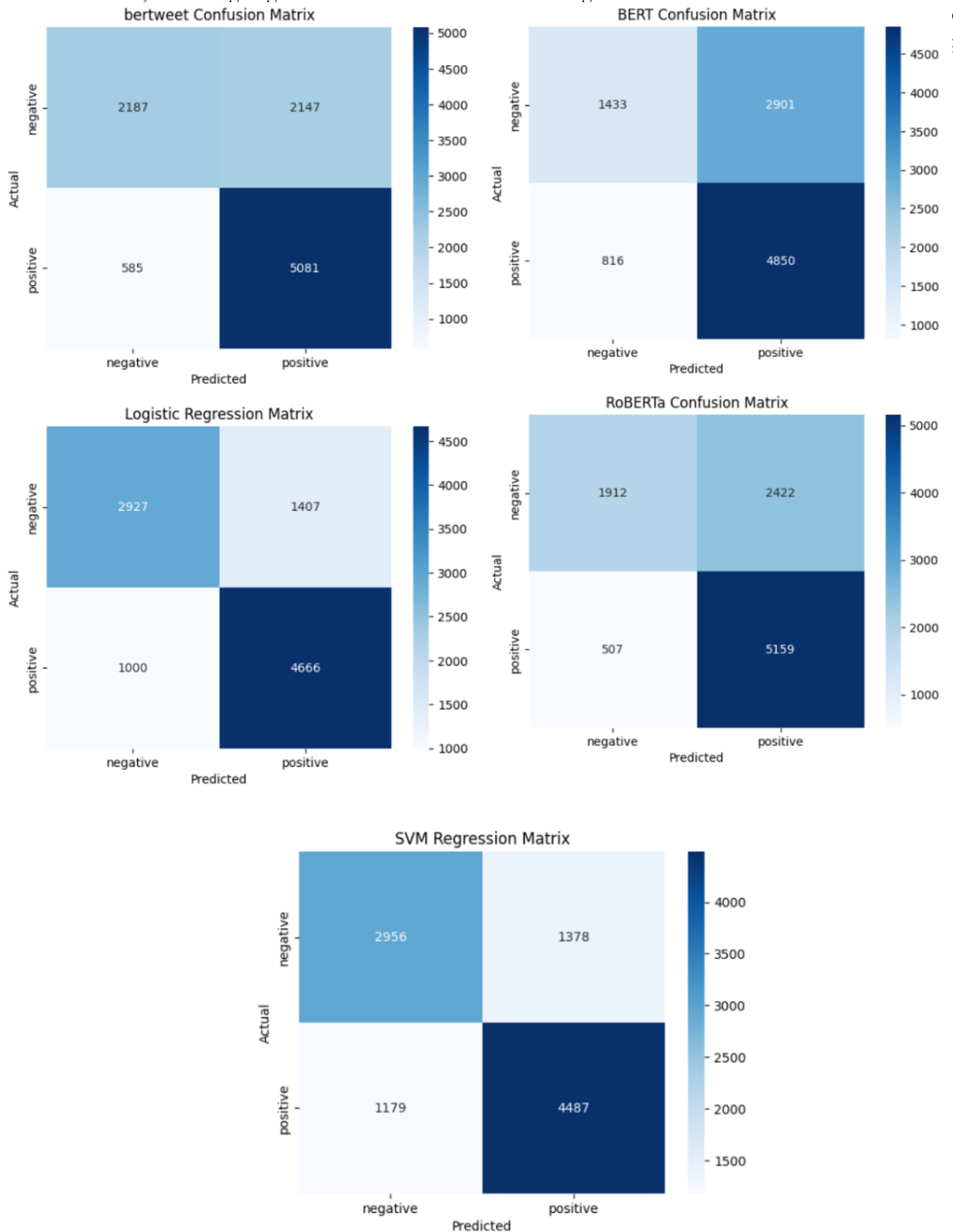


Figure 14 Confusion matrix for models

The classification report results show clear differences between the models when comparing precision, recall, and F1-score. Logistic Regression achieved the most balanced performance across both sentiment classes, with strong results for the positive class and stable macro and weighted averages. SVM performed similarly but showed lower recall for positive sentiment, meaning it missed more positive tweets while being slightly better at identifying negative sentiment. Among the transformer models, RoBERTa and BERTweet showed stronger recall for positive sentiment, indicating better detection of positive tweets, but they also produced more false positives for the negative class. BERT showed the weakest overall performance, with lower balance between precision and recall. Overall, Logistic Regression provided the most consistent results, while RoBERTa and BERTweet demonstrated stronger contextual understanding for positive sentiment.

Based on the ROC curve results, there are clear differences in how well each model separates positive and negative sentiment across different classification thresholds. TF-IDF with Logistic Regression showed strong separation between the two classes, achieving a high AUC value of 0.844, which indicates strong overall classification performance and reliable ability to distinguish positive tweets from negative ones. BERTweet achieved a moderate AUC value of 0.701, showing that it performed better than random classification and was able to distinguish sentiment reasonably well, but not as strongly as Logistic Regression, mainly due to a higher number of false positive predictions. In contrast, BERT produced a low AUC value of 0.481, which is close to random classification, indicating weak performance in separating sentiment classes in this implementation. Overall, the ROC analysis confirms that TF-IDF with Logistic Regression achieved the strongest performance, followed by BERTweet, while BERT showed the weakest performance among the evaluated models. An AUC value closer to 1 indicates that the model is better at distinguishing between positive and negative sentiment, while a value close to 0.5 indicates performance close to random classification Figure 15.

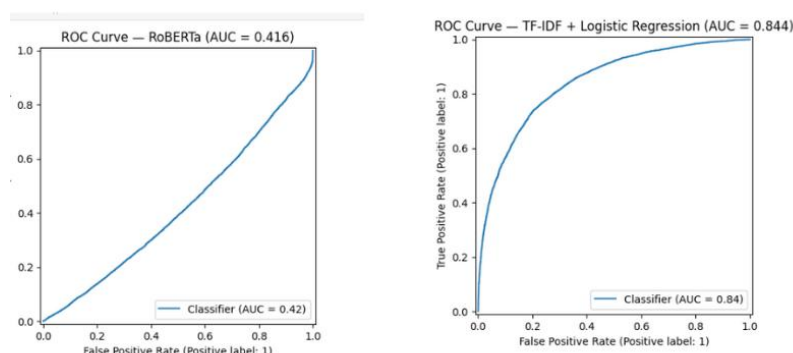


Figure 15 ROC curves illustrating the classification performance of TF-IDF + Logistic Regression and RoBERTa.

Inferences

From the experimental results, it can be inferred that traditional machine learning models such as Logistic Regression and SVM perform strongly for tweet sentiment analysis when combined with TF-IDF features. Logistic Regression showed more balanced performance and achieved a higher F1-score, making it effective for identifying both positive and negative sentiment. SVM performed slightly better in identifying negative sentiment but missed more positive cases. Transformer-based models demonstrated stronger ability to capture contextual meaning in tweets, especially for positive sentiment. Among them, RoBERTa provided the most balanced results, while BERTweet showed high sensitivity to positive sentiment due to its Twitter-specific pretraining. Overall, the results confirm that transformer models are more powerful in capturing context, while traditional models remain competitive and efficient for large-scale sentiment classification.

Recommendations

For future work, it is recommended to use GPU-enabled environments to fine-tune transformer models such as BERT, RoBERTa, and BERTweet, which is expected to significantly improve performance. Additional techniques such as class imbalance handling, threshold tuning, and hyperparameter optimization could further improve results. Exploring hybrid approaches that combine TF-IDF features with deep learning models may also enhance sentiment classification performance. Finally, extending the task to multi-class sentiment analysis or aspect-based sentiment analysis would provide deeper insights into public opinion on Twitter.

Reflection

One of the main challenges encountered during this project was the limitation of computational resources. Processing the full dataset and running transformer-based models on a CPU significantly increased execution time and restricted the ability to fine-tune models or perform extensive experimentation. Despite this limitation, the project successfully implemented and evaluated multiple models using the complete dataset. This experience improved understanding of practical NLP pipelines, model selection, and the trade-offs between performance and computational cost in real-world applications.

Conclusion

This evaluation demonstrates that both traditional machine learning models and transformer-based models are effective for tweet sentiment analysis, each with distinct strengths. Logistic Regression and SVM achieved strong and balanced performance, with F1-scores exceeding the target threshold of 70%, confirming their suitability for TF-IDF-based sentiment classification. Among the transformer-based models, RoBERTa and BERTweet showed strong capability in identifying positive sentiment, with RoBERTa providing more balanced performance across both classes. Overall, the project successfully achieved its objective of building and evaluating multiple sentiment classifiers while meeting the target F1-score requirement and highlighting practical challenges and real-world constraints related to computational resources.

Github Link

- <https://github.com/Enasnayel/Tweet-Sentiment-Analysis>

Dataset Link

- <https://huggingface.co/datasets/gxb912/large-twitter-tweets-sentiment>
- <https://drive.usercontent.google.com/download?id=1PSXWlvqeLRPNpEfJdZJUoVXJ1Qz5ewAu&export=download&authuser=0>

REFERENCE

- [1] G. Xiangbo, "gxb912/large-twitter-tweets-sentiment." [Online]. Available: <https://huggingface.co/datasets/gxb912/large-twitter-tweets-sentiment>
- [2] A. Albladi, M. Islam, and C. Seals, "Sentiment Analysis of Twitter Data Using NLP Models: A Comprehensive Review," *IEEE Access*, vol. 13, pp. 30444–30468, 2025, doi: 10.1109/ACCESS.2025.3541494.
- [3] "Slang." [Online]. Available: <https://drive.usercontent.google.com/download?id=1PSXWlvqeLRPNpEfJdZJUoVXJ1Qz5ewAu&export=download&authuser=0>
- [4] N. S. Wardana, F. P. Aditiawan, and A. P. Sari, "Logistic Regression Classification with TF-IDF and FastText for Sentiment Analysis of LinkedIn Reviews," *VISA J. Vis. Ideas*, vol. 4, no. 3, Aug. 2024, doi: 10.47467/visa.v4i3.2835.
- [5] N. Z. B. Jannah and K. Kusnawi, "Comparison of Naïve Bayes and SVM in Sentiment Analysis of Product Reviews on Marketplaces," *Sinkron*, vol. 8, no. 2, pp. 727–733, Mar. 2024, doi: 10.33395/sinkron.v8i2.13559.
- [6] K. L. Tan, C. P. Lee, and K. M. Lim, "RoBERTa-GRU: A Hybrid Deep Learning Model for Enhanced Sentiment Analysis," *Appl. Sci.*, vol. 13, no. 6, p. 3915, Mar. 2023, doi: 10.3390/app13063915.
- [7] A. Bello, S.-C. Ng, and M.-F. Leung, "A BERT Framework to Sentiment Analysis of Tweets," *Sensors*, vol. 23, no. 1, p. 506, Jan. 2023, doi: 10.3390/s23010506.
- [8] M. S. Sayeed, V. Mohan, and K. S. Muthu, "BERT: A Review of Applications in Sentiment Analysis," *HighTech Innov. J.*, vol. 4, no. 2, pp. 453–462, Jun. 2023, doi: 10.28991/HIJ-2023-04-02-015.